AI Model Gateway Customer Guide

A simple reference for customers who want a multi-model AI platform direction, but may not have a large technical team yet.

Audience	Business and product teams with basic technical awareness
Prototype	Local AI Model Gateway with Qwen / DashScope first
Main idea	One customer API, many model providers behind the scenes
Demo mode	Mock mode first, live Qwen mode only when needed

Executive Summary

An AI Model Gateway lets customers call one simple API while the platform manages model routing, provider differences, usage control, and future fallback logic. The first prototype uses Alibaba Cloud Model Studio / Qwen because it is available now and keeps testing cost low.

- Start with Qwen and mock mode before adding more paid providers.
- Use an OpenAI-compatible API so customers can understand the integration quickly.
- Show routing visually: smart-fast is a customer-facing name that maps to qwen-plus.
- Keep enterprise API Gateway features optional until customer demand is clear.

Layered Architecture

Layer	What It Means	Example
1. Customer Systems	The customer's app, backend, or agent	Website, chatbot, internal tool
2. One Simple API	One standard API format for customers	/v1/chat/completions
3. Gateway Entry Control	Controls who can call and how much they can use	API keys, limits, logs
4. Model Control	Chooses the public model and route	smart-fast -> qwen-plus
5. Provider Adapters	Translates request and response formats	DashScope adapter
6. Model Providers	The real upstream AI model provider	Alibaba Cloud Qwen

The dashboard also shows this as a visual architecture map, so non-technical customers can follow the layers from customer systems to model providers.

How Route Simulation Works

The dashboard explains routing without spending money. In mock mode, the gateway reads the requested model, checks the registry, maps the public model to a real upstream model, and returns a simulated response with a route trace.

```
Customer request: model = smart-fast
Registry lookup: smart-fast maps to qwen-plus
Provider adapter: prepare DashScope-compatible request
```

```
Mock response: show the route trace without calling Alibaba Cloud
```

What The Current Prototype Includes

- Visual dashboard at the gateway root URL.
- OpenAPI contract at /openapi.json for customer technical handoff.
- Postman collection at /postman_collection.json for click-through demos.
- Demo bundle manifest at /v1/gateway/demo-bundle for customer presentation flow.
- Handoff checklist endpoint for customer-shareable links, admin-only material, meeting checks, and follow-up actions.
- Customer integration guide at /v1/gateway/integration-guide with safe code examples.
- OpenAI-compatible /v1/chat/completions endpoint.
- Customer API key authentication with Bearer token.
- Basic in-memory request limit.
- Model registry in model_registry.json.
- SQLite request and usage records for restart-safe demo data.
- Provider, customer, model, and request summary endpoints.
- Operational alerts endpoint with simple next steps.
- Incident playbook endpoint with support scenarios and customer-safe wording.
- Support policy endpoint for prototype, pilot, and production support stages.
- Pilot checklist endpoint for before, during, and after a customer trial.
- Pilot scorecard endpoint for deciding discovery, extended pilot, or production hardening.
- Discovery checklist endpoint for customer goals, provider scope, governance, reporting, and production expectations.
- Proposal summary endpoint for customer-facing pilot scope, exclusions, risks, and next steps.
- Deployment readiness endpoint for environment, preflight checks, operations, rollback, and deployment options.
- Migration plan endpoint for phased customer cutover, checklist, rollback, and evidence.
- Production backlog endpoint for prioritized P0, P1, and P2 hardening tasks.
- Executive brief endpoint for non-technical customer stakeholders.
- Roadmap endpoint for prototype, pilot, production hardening, and multi-provider expansion.
- Decision guide endpoint for API Gateway, managed AI Gateway, custom Model Gateway, and OpenRouter-like options.
- FAQ endpoint for common customer questions and objections.
- Demo script endpoint for a 15 minute customer walkthrough.
- Access matrix endpoint for customer and model permissions.
- Provider health endpoint for mock-ready, live-ready, degraded, and not-ready states.
- Provider create, update, and disable actions for provider lifecycle demos.
- Provider contract matrix for adapter differences across Qwen, OpenAI-compatible APIs, Claude, and planned providers.

- Model catalog endpoint for provider status, fallback chain, usage, and pricing metadata.
- Model route create, update, and disable actions for route lifecycle demos.
- Route preview endpoint to dry-run model access, budget, provider readiness, and fallback order.
- Cost estimate endpoint to dry-run token estimate, estimated cost, and budget impact.
- Customer reports endpoint for request count, errors, token usage, and budget state.
- Commercial policy endpoint for pricing assumptions, budget behavior, invoice preview limits, and exclusions.
- Procurement pack endpoint for vendor review tracks, evidence documents, approval owners, and red lines.
- Request activity endpoint with simple filters for troubleshooting.
- Request detail lookup using gateway.request_id returned in chat responses.
- Config check endpoint for demo keys and missing production settings.
- Production readiness endpoint for plain-English go-live gaps.
- Data governance endpoint for prompt handling, logs, retention, customer keys, and provider secrets.
- Security review endpoint for threat model, current controls, production controls, and go-live security gates.
- Operations runbook endpoint for SLO-style targets, daily checks, alert actions, and owners.
- Evaluation plan endpoint for model quality, reliability, latency, cost, safety, fallback, and scorecards.
- Structured route decision summary for support explanations.
- Local safety preview for obvious sensitive data.
- Customer key issue preview for safe onboarding demos.
- Customer key create, rotate, and disable actions for lifecycle demos.
- Audit events for customer key lifecycle actions.
- Customer default policy for automatic routing presets.
- Invoice preview with JSON and CSV output.
- Capability routing control for streaming and tools.
- Request-level fallback controls for routing demos.
- Request-level provider allow-list for routing control demos.
- Request-level route strategy for registry, lowest cost, fastest, and healthiest routing.
- Named policy presets for common routing controls.
- Mock OpenAI-style tool call response for agent demos.
- Simple customer plans with token and cost budgets.
- Bring Your Own Key mapping through environment variables.
- Provider adapter scaffolds for OpenAI-compatible APIs and Claude-style APIs.
- Automated mock regression test that does not spend provider credits.
- Mock mode for demos and planning.
- Live Qwen mode when DASHSCOPE_API_KEY is available.

Main Technical Difficulties

Difficulty	Why It Matters	Prototype Status
Provider format differences	Each provider may use different request and response details.	Handled first for DashScope-compatible chat
API handoff	Customer technical teams need a clear API contract.	OpenAPI contract endpoint
Demo handoff	Customer technical teams may want to click through requests.	Postman collection endpoint
Demo flow	Business and technical users need to know what to look at first.	Demo bundle manifest endpoint
Customer handoff	Teams need to know what can be shared and what stays admin-only.	Handoff checklist endpoint
Customer integration	A customer technical team needs safe examples for their own key and models.	Customer integration guide endpoint
Streaming	Chat UIs and agents often need incremental tokens.	Mock streaming included; live provider differences
Tool calling	OpenAI, Claude, and Qwen may differ in tool format.	Mock tool_calls plus basic normalization
Token usage	Billing and quota require reliable usage data.	Stored in SQLite
Operational alerts	Non-technical users need a clear action list.	Alerts with severity, area, and next step
Incident response	Support teams need a shared script when requests fail.	Playbook with signals, steps, and customer wordi
Support policy	Customers need to know what support is promised at each stage.	Prototype, pilot, and production support guide
Pilot planning	A customer trial needs scope, roles, success criteria, and exit decision.	Pilot checklist endpoint
Pilot scoring	Teams need to know whether the pilot worked.	Scorecard with score, decision, weak criteria, and
Customer discovery	Broad gateway ideas need clear scope before implementation.	Discovery checklist with questions, evidence, and
Proposal summary	Customers need a simple scope note after discovery.	Customer-facing pilot scope, exclusions, risks, an
Executive communication	Non-technical stakeholders need a short business summary.	Executive brief endpoint
Roadmap planning	Customers need to see the path from demo to production.	Roadmap endpoint
Build or buy decision	Customers need to compare gateway options clearly.	Decision guide endpoint
Customer objections	Sales and support need consistent answers to common questions.	FAQ endpoint
Demo delivery	A presenter needs a simple meeting flow, talk track, and likely questions.	Demo script endpoint
Production readiness	Customers need to understand why demo-ready is not production-ready.	Plain-English readiness report
Deployment readiness	Customer technical teams need to know what must be configured before running it.	Environment, preflight, health checks, rollback, an
Customer migration	Customers need a low-risk path from direct provider calls to the gateway.	Phased cutover plan and rollback checklist
Production backlog	Teams need to convert gaps into fundable engineering tasks.	P0, P1, and P2 hardening backlog
Data governance	Customers need to know what happens to prompts, logs, retention, and secrets.	Plain-English governance review
Security review	Customers will ask what can go wrong and what controls exist.	Threat model, current controls, and go-live gates

Operations runbook	Customers need to know who watches the gateway after a pilot starts.	SLO-style checks, owners, and alert actions
Model evaluation	Customers will ask how the gateway chooses a better model.	Evaluation plan, sample prompts, and scorecards
Access control	Each customer may be allowed to use different models.	Access matrix by customer and model
Provider health	Customers need to know whether a provider can serve traffic.	Readiness view based on config and recent traffic
Provider lifecycle	Teams need to add and stop providers safely.	Local JSON-backed create, update, and disable a
Provider contracts	Teams need to see why each provider needs adapter tests.	Contract matrix for provider differences
Model catalog	Customers need to understand public model names and routes.	Catalog with provider status, fallback chain, usage
Model route lifecycle	Teams need to change public model routes safely.	Local JSON-backed create, update, and disable a
Route preview	Sales and support need to explain a route before spending credits.	Dry-run endpoint and dashboard preview
Route decision summary	Customers may ask why a route was chosen.	Plain-language summary and reasons
Safety preview	Customers may worry about secrets in prompts.	Local preview for obvious emails, phone numbers
Provider routing control	Some customers may want only approved providers for a request.	gateway_allowed_providers filters route candidate
Route strategy	Customers may want cost, latency, or health-aware routing.	Request-level route strategy reorders candidates
Policy presets	Non-technical users need simple policy names.	gateway_policy applies named routing presets
Customer default policy	Customers may not want to send routing controls every time.	Customer config can set a default policy
Capability routing	Requests may need streaming, tools, or other abilities.	gateway_required_capabilities filters route candid
Cost estimate	Customers need budget planning before live calls.	Dry-run estimate for tokens, cost, and budget imp
Commercial policy	Customers need to know what is an estimate and what requires a contract.	Pricing assumptions, budget rules, and exclusions
Procurement pack	Customers may need procurement, legal, IT, security, and finance review.	Review tracker, evidence documents, approval ow
Customer onboarding	New customers need keys, limits, and model access.	Key issue preview creates a safe config snippet
Key lifecycle	Customers need key rotation and disable workflows.	Local JSON-backed create, rotate, and disable ac
Audit trail	Teams need to know who changed customer access.	Audit events for customer lifecycle actions
Customer self-service	Customers need to see their own access, usage, and budget.	Customer self view with no provider secret exposu
Invoice preview	Customers need a simple billing story.	Estimated invoice preview with CSV export
Customer reporting	Customers need a simple usage and budget story.	Per-customer report endpoint and dashboard card
Request troubleshooting	Support teams need to see what happened to a request.	Filtered request activity feed
Request detail	Support teams need one-request lookup.	gateway.request_id and request detail endpoint
Fallback	Retrying another model needs clear business rules.	Registry fallback plus request-level controls
Customer key management	Each customer needs limits, logs, and access control.	Minimum version plus BYOK mapping

Cost control	Different models have different prices and limits.	Simple token and cost budgets included

Why The Test Script Matters

The repository includes test_gateway.py. It starts the gateway in mock mode and checks API keys, model listing, route tracing, fallback, model access rules, token budget blocking, SQLite logs, and status output without leaking provider keys.

API Contract

/openapi.json exposes an OpenAPI contract for the prototype. It describes the customer API, admin control-plane API, bearer key security schemes, and the main endpoint groups. /postman_collection.json exposes a ready-to-import Postman collection with base_url, gateway_api_key, and admin_api_key variables. /v1/gateway/demo-bundle lists the dashboard, customer self view, OpenAPI contract, Postman collection, PDF guide, recommended demo order, safe curl examples, and production notes. /v1/gateway/handoff-checklist explains what can be shared with customers, what should stay admin-only, meeting checks, follow-up actions, and the rule that provider API keys stay private. The dashboard also has a Customer handoff package section that links the main artifacts in one place and an Integration command starter with local curl commands. /v1/gateway/integration-guide lets a customer use their own key to see allowed models, a recommended first model, curl, Python, JavaScript, streaming examples, and a go-live checklist without exposing provider secrets. /v1/gateway/sdk-starter adds a .env template, starter Python and JavaScript files, first-run commands, common error fixes, and a handoff checklist. These help a business team understand the story first and help a customer technical team import or click through the prototype quickly.

Handoff Checklist

/v1/gateway/handoff-checklist is an admin-only customer meeting checklist. It separates customer-shareable material from internal readiness material. It covers the business overview, customer technical handoff, pilot decision package, internal readiness package, meeting checks, and follow-up actions.

Group	What It Means
Business overview	Links for non-technical customer explanation.
Customer technical handoff	API contract, Postman, integration guide, and SDK starter.
Pilot decision package	Pilot checklist, scorecard, customer reports, and success summary.
Internal readiness package	Production readiness, deployment readiness, backlog, change plan, and data governance.

Admin Summary Endpoints

The prototype exposes local admin JSON views for provider status, operational alerts, access matrix, provider health, model catalog, route preview, customer reports, request activity, request detail lookup, usage by customer, usage by model, and request summaries. These make the control layer easier to explain. The local demo uses a separate admin key. In production, this key should be changed and protected.

Operational Alerts

/v1/gateway/alerts combines config warnings, provider readiness, customer budget state, and recent request errors. Each alert includes severity, area, message, and a simple next step. This helps non-technical users understand what

needs attention before a live demo.

Incident Playbook

/v1/gateway/incident-playbook explains common failure scenarios in simple English. It covers provider readiness, recent request errors, customer budget blocks, demo key or secret risks, and provider contract gaps. Each scenario includes triggers, signals to check, current evidence, operator steps, and customer-safe wording. It is not a legal SLA, but it helps sales, support, and technical teams explain issues consistently.

Support Policy

/v1/gateway/support-policy explains prototype, technical pilot, and production target support expectations. It defines P1, P2, and P3 severity, first checks, escalation path, and customer-safe wording. It clearly says the current prototype is not a legal production SLA. This helps customers understand what can be promised now and what needs a real contract later.

Pilot Checklist

/v1/gateway/pilot-checklist helps prepare a small customer trial. It covers recommended scope, roles, before-pilot checks, during-pilot checks, after-pilot review, success criteria, and the stop, extend, or productionize decision. It helps keep the pilot small, honest, and measurable.

Onboarding Plan

/v1/gateway/onboarding-plan turns the demo into a simple customer pilot path. It explains Day 0 customer alignment, Day 1 safe access setup, Day 2 mock technical test, Day 3 route and provider review, Day 4 usage, cost, and support review, and Day 5 pilot decision. Each step has an owner, actions, evidence, and an exit check, so non-technical customers can understand what happens after the demo.

Executive Brief

/v1/gateway/executive-brief gives a short business summary for non-technical stakeholders. It explains the one-sentence value, why the gateway matters, what the demo proves, what is not production-ready yet, the recommended customer story, pilot recommendation, support and readiness position, and next step.

Roadmap Endpoint

/v1/gateway/roadmap explains the path from prototype to production. It includes prototype explanation, technical pilot, production hardening, and multi-provider expansion. Each phase has a goal, deliverables, exit criteria, and main risks. This helps customers understand next steps without jumping too quickly into production promises.

Decision Guide

/v1/gateway/decision-guide compares normal API Gateway, managed AI Gateway, custom Model Gateway, and OpenRouter-like marketplace options. The dashboard also renders this as Gateway options comparison cards. It explains what each option is good at, what it does not solve, and when to choose it. This directly answers the early question: is an API Gateway enough?

Customer FAQ

/v1/gateway/faq answers common customer questions about API Gateway, Qwen, OpenRouter, prompt and key exposure, cost control, provider failure, adding more providers, pilot next steps, and production readiness. It helps

sales, support, and technical teams answer in the same simple language.

Demo Script

/v1/gateway/demo-script gives a 15 minute customer walkthrough. The dashboard also shows this as Presenter mode. It tells the presenter what to say first, which endpoint to show, how to explain routing, how to answer common objections, and how to close with a small pilot. This helps non-technical customers understand the project without reading code.

Customer Self View

/v1/gateway/me lets a customer use their own gateway key to see their own plan, allowed models, budget state, usage, recent requests, and invoice preview. It does not expose raw customer API keys or provider API keys. This is useful when the customer asks: what can I use, and how much have I used?

Access Matrix

/v1/gateway/access-matrix shows which customers can use which public models, why access is allowed or blocked, the customer's budget state, and the provider readiness for each model. This helps explain customer-level API control.

Provider Health

/v1/gateway/provider-health answers a simple question: can this provider serve traffic now? It checks enabled models, live key readiness, customer BYOK readiness, recent errors, and average latency. In mock mode, ready_mock means the provider can be explained without spending credits. It does not mean the live provider key is ready.

Provider Lifecycle

The prototype can create, update, and disable providers. The dashboard has a Provider lifecycle panel for these demos. A provider defines the upstream API type, base URL, and API key environment variable name. Disabling a provider also disables active model routes that point to it, so the local registry stays valid. These admin actions update model_registry.json, reload the local runtime, and write audit events. The prototype stores the provider key environment variable name, not the provider secret value. Production should use a database, secret manager, approval workflow, readiness checks, and rollout controls.

Provider Contract Matrix

/v1/gateway/provider-contracts explains why an AI Model Gateway is more than a normal API Gateway. It compares OpenAI-compatible providers, Alibaba Cloud Model Studio compatible mode, Anthropic Claude, and planned Xiaomi or other local model providers. It shows auth, endpoint path, request shape, response shape, streaming, tool calling, usage fields, adapter status, and remaining contract gaps.

Model Catalog

/v1/gateway/model-catalog explains what each public model name means. It shows the upstream model, provider status, fallback chain, capabilities, pricing metadata, request count, errors, tokens, and estimated cost. This helps customers understand that smart-fast can be a simple public name while the gateway manages the real provider route behind it.

Model Route Lifecycle

The prototype can create, update, and disable public model routes. The dashboard has a Model route lifecycle panel for these demos. A route defines the customer-facing model name, provider, upstream model, fallback models, capabilities, and pricing metadata. These admin actions update model_registry.json, reload the local runtime, and write audit events. Production should add approval workflow, version history, rollback, and staged rollout.

Route Preview

/v1/gateway/route-preview is a dry run. It does not call the provider. It shows whether a customer can use a model, whether budget is available, which model is primary, which models are fallback routes, which provider would handle the request, and whether that provider is ready.

Route Decision Summary

Route preview and chat responses include route_decision. It explains the requested public model, selected upstream model, provider, fallback policy, provider controls, capability controls, and simple reasons for the routing decision. This is useful when a customer asks why a request used a certain model or provider.

Safety Preview

/v1/gateway/safety-preview checks for obvious sensitive data before a provider call. It can detect examples such as email addresses, possible phone numbers, possible API keys, and secret assignments. Chat requests can also use gateway_safety_check to return safety metadata, gateway_block_sensitive to block detected sensitive data, or gateway_redact_sensitive to replace detected sensitive data before the provider call. This is useful for explaining gateway guardrails, but it is not a full DLP or compliance system.

Provider Routing Control

The prototype supports gateway_allowed_providers. This lets a request say: only use these providers for this route. For example, route preview can allow only dashscope. If no route matches the allowed provider list, the gateway returns a clear error. This helps customers understand that the gateway is a control layer, not only a normal API proxy.

Routing Strategy

The prototype supports gateway_route_strategy. Supported values are registry, lowest_cost, fastest, and healthiest. The strategy reorders route candidates before the provider call. Route preview and chat responses include candidate_scores in route_decision. This helps explain cost-aware, latency-aware, and health-aware routing. Production should use stronger cost data, latency windows, provider SLAs, and customer policy rules.

Policy Presets

The prototype supports gateway_policy. A policy preset is a short name for common routing controls. Current presets include balanced, lowest_cost, fastest, and tool_ready. /v1/gateway/policy-presets lists the presets. Explicit request controls override preset controls. This helps non-technical customers choose a simple policy name instead of many technical fields.

Customer Default Policy

A customer can have default_policy in customer_keys.json. If a request does not send gateway_policy, the gateway uses the customer default. If a request sends gateway_policy, the request value wins. This lets a customer use one

gateway key and one default routing behavior without sending routing controls every time.

Capability Routing Control

The prototype supports gateway_required_capabilities. This lets route preview or a real request require abilities such as streaming or tools. stream=true requires streaming. A tools request requires tools. If no route supports the required ability, the gateway returns a clear no_capability_route error. This explains why model metadata matters in a real gateway.

Cost Estimate

/v1/gateway/cost-estimate is a dry run. It does not call the provider. It estimates prompt tokens, completion tokens, estimated cost, and remaining budget after the estimate. This is useful for planning, but real provider token usage can differ.

Customer Key Issue Preview

/v1/gateway/key-issue-preview creates a safe customer onboarding package. It returns a generated gateway API key, a masked key for display, a customer config snippet, allowed models, limits, and next steps. It does not save the customer. In production, this would connect to a real customer database and secret manager.

Customer Key Lifecycle

The prototype can also create a customer, rotate a customer gateway key, and disable a customer. The dashboard has a Customer lifecycle panel for these demos. These admin actions update customer_keys.json and reload the local runtime. After rotation, the old key stops working. After disable, the customer key stops working. This is useful for demos, but production should use a database, audit logs, approval workflow, and a secret manager.

Audit Events

/v1/gateway/audit-events records admin lifecycle actions such as customer created, key rotated, customer disabled, provider changed, and model route changed. The dashboard also has an Audit timeline section for recent changes. Each event includes actor, action, target customer, provider, or model route, time, and safe details such as masked keys and key hashes. It does not store full gateway API keys. This helps explain control history, but it is not a full compliance audit system.

Commercial Policy

/v1/gateway/commercial-policy explains how to talk about pricing, budgets, invoice previews, and commercial boundaries before a real contract exists. It covers what can be shown now, what needs contract approval later, request limit behavior, token budget behavior, cost budget behavior, invoice preview limits, approval questions, and prototype exclusions. It is not a legal quote, tax invoice, payment system, or audited billing ledger.

Area	Plain-English meaning
What can be shown now	Estimated usage, estimated cost, budgets, invoice preview, and CSV export.
Contract later	Final pricing, payment terms, overage behavior, refunds, tax, and legal invoice fields.
Budget behavior	Request, token, and cost limits can block or warn in the prototype.
Exclusions	No legal quote, payment collection, refund workflow, or audited billing ledger yet.

Procurement Pack

/v1/gateway/procurement-pack helps a customer share the idea with procurement, legal, IT, security, and finance before a pilot or purchase discussion. It explains what the prototype is, what it is not, review tracks, evidence documents, approval owners, red lines, and meeting questions. It does not replace a signed contract, legal security attestation, production SLA, final price quote, or tax invoice.

Area	Plain-English meaning
Review tracks	Business, IT, security, data, finance, legal, and customer technical teams know what to check.
Evidence documents	The pack points to OpenAPI, Postman, security review, data governance, commercial policy, and
Approval owners	Each owner knows what they must approve before production.
Red lines	The team knows which promises should wait for production approval.

Invoice Preview

/v1/gateway/invoice-preview uses local usage records to create an estimated billing preview. It shows requests, errors, prompt tokens, completion tokens, total tokens, estimated cost, remaining budget, usage by model, and usage by provider. It can return JSON or CSV. It is not a legal invoice, but it helps explain how customer billing reports could work later.

Customer Reports

/v1/gateway/customer-reports shows one report per customer. It includes request count, error count, token usage, remaining budget, models used, providers used, and recent requests. This helps explain that a gateway is also a control and reporting layer, not only a model router.

Customer Success Summary

/v1/gateway/customer-success turns customer usage data into account health. It shows healthy, watch, and at-risk customer counts, health status per customer, risk reasons, recommended follow-up action, simple business metrics, meeting questions, and evidence endpoints for support. This helps sales, support, and customer success teams decide who needs attention before the next customer call.

Pilot Scorecard

/v1/gateway/pilot-scorecard helps decide whether a customer pilot should stay in discovery, continue testing, or move toward production hardening. It scores first request completion, request tracing, error rate, budget state, model access, and production gap acknowledgement. Each customer gets a score, decision, metrics, weak criteria, and next action.

Request Activity

/v1/gateway/request-activity shows recent gateway decisions. It can filter by customer, model, provider, error code, status, and limit. This helps explain which customer sent a request, which public model was requested, which upstream model was used, which provider handled it, and whether the request succeeded or failed.

Request Detail

Every successful chat response includes gateway.request_id. /v1/gateway/request-detail can look up one request by that id. It shows the customer, public model, upstream model, provider, outcome, latency, and nearby usage record when available.

Config Check

The prototype includes /v1/gateway/config-check. It warns about demo admin keys, demo customer keys, missing live provider keys, and direct secret values in local JSON files. This is not a full security audit, but it is a useful readiness checklist for customer demos.

Production Readiness Report

/v1/gateway/production-readiness explains go-live gaps in plain English. The dashboard also shows this as Production readiness cards. It groups the work into security, provider readiness, customer controls, routing and fallback, observability, billing, and documentation handoff. Each area has a status, evidence, and next step. This helps a customer understand why a prototype can be demo-ready but still not production-ready.

Deployment Readiness Guide

/v1/gateway/deployment-readiness explains how to move from local demo to a controlled pilot or production deployment. It covers local demo, live Qwen test, technical pilot, and production target stages. It also lists required environment variables and files, preflight checks, operational health checks, rollback plan, and deployment options. It is not a one-command production deploy.

Area	What it explains
Stages	Local demo, live Qwen test, technical pilot, and production target.
Environment	Admin key, customer keys, DASHSCOPE_API_KEY, registry, customer config, and logs.
Preflight	Admin key, provider key strategy, customer keys, data policy, and readiness blockers.
Operations	Health check, model list, mock chat, route preview, and provider health.
Rollback	Config versioning, audit events, database rollback, and fallback route plan.

Customer Migration Plan

/v1/gateway/migration-plan explains how a customer can move from direct model provider calls to one gateway API without switching everything at once. It covers current-state discovery, shadow gateway setup, mock and evaluation testing, limited live pilot, gradual cutover, production decision, cutover checklist, rollback plan, and evidence endpoints. It is a cutover plan for customer conversations, not a one-click migration tool.

Phase	Purpose
Current-state discovery	Understand current provider calls, prompts, models, owners, and data rules.
Shadow gateway setup	Create public model names and customer key without changing production traffic.
Mock and evaluation test	Show routing, fallback, cost estimate, safety, and model evaluation evidence.

Limited live pilot	Send small approved traffic through the gateway while old provider path remains available.
Gradual cutover	Increase traffic only after error, latency, budget, and support checks pass.
Production decision	Choose stop, extend pilot, or fund production hardening.

Production Hardening Backlog

/v1/gateway/production-backlog turns prototype gaps into prioritized engineering work. It groups work into P0 production blockers, P1 pilot and limited-production hardening, and P2 scale or marketplace expansion. It covers secrets, storage, data governance, operations, change control, provider contracts, billing, tenant controls, marketplace planning, and deployment automation.

Priority	Meaning
P0	Must be done before production customer traffic.
P1	Needed for a serious pilot or limited production.
P2	Useful for scale, automation, or later multi-provider expansion.

Production Launch Plan

/v1/gateway/launch-plan converts readiness gaps into go-live gates. It explains security and secrets, provider live readiness, customer access and budgets, routing and fallback, support and observability, billing and commercial rules, and customer handoff. Each gate has an owner, required evidence, current evidence, approval question, status, and next step. It also shows required signoffs and rollout stages from internal live test to broader rollout.

Change Management Plan

/v1/gateway/change-management explains how to change customers, providers, or model routes without surprising a customer. It includes change types, risk level, approval owner, before-change checklist, after-change validation, rollback path, and evidence endpoints. It is not automatic rollback yet, but it is a simple operating plan for safer demos, pilots, and production discussions.

Data Governance Review

/v1/gateway/data-governance explains what happens to prompt data, response data, logs, customer gateway keys, and provider secrets. It covers prompt handling, provider secret handling, customer key handling, logging and retention, sensitive data preview, customer visibility, and production gaps. It is not a compliance certification. It is a simple checklist for customer trust discussions before real production traffic.

Question	Why it matters
Can prompts be logged?	Support teams need traces, but prompts may contain sensitive customer data.
How long are logs kept?	Customers need retention and deletion rules before production use.
Who can see request details?	Admin access must be limited and auditable.

Where are provider keys stored?	Provider secrets must live in a secret manager, not customer-facing JSON.
What sensitive data is blocked or redacted?	The gateway needs clear customer-specific data handling rules.

Security Review

/v1/gateway/security-review explains the main security questions a customer will ask before trusting one gateway with many AI providers. It covers customer gateway key risk, provider secret risk, sensitive prompt data in logs, wrong customer or model access, unsafe admin changes, current prototype controls, production controls still needed, and go-live security gates. It is a simple threat model for customer conversations, not a penetration test, SOC 2 report, legal compliance review, or production approval.

Risk	Current prototype control	Production control needed
Customer key leak	Allowed models, request limits, budgets, rotate and disable actions.	Real key storage, expiry, approvals, emergency disable, and alerts
Provider secret exposure	Provider records use environment variable names and mask secret-like fields.	Secret manager integration, rotation schedules, restricted operators, and se
Sensitive prompt data in logs	Safety preview can show obvious sensitive patterns.	Retention rules, masking, deletion workflow, and customer-spec
Wrong model access	Customer allowed_models and access matrix.	Tenant isolation tests, export controls, and stronger policy revie
Unsafe admin change	Lifecycle endpoints create audit events and change plan explains safe rollout.	Approval workflows, staged rollout, config history, and automated

Operations Runbook

/v1/gateway/operations-runbook explains what to watch after the gateway is used by a customer, who should act, and what evidence to collect before changing routes or promises. It covers SLO-style targets, daily checks, alert actions, owner responsibilities, current signals, and evidence endpoints. It is a simple operations runbook for customer pilots, not a legal SLA or full monitoring system.

Area	What it means
SLO-style targets	Track health, error rate, latency, and budget before promising legal SLA terms.
Daily checks	Review health, provider status, customer errors, budgets, and security alerts.
Alert actions	Use first actions and customer-safe wording for provider, request, budget, and security signals.
Ownership	Name support, gateway, platform, and business owners before a pilot starts.
Evidence	Use alerts, provider health, request detail, customer reports, invoice preview, and support policy.

Model Evaluation Plan

/v1/gateway/evaluation-plan explains how to compare models before routing real customer traffic. It covers task quality, reliability, latency, cost, safety and data handling, fallback behavior, sample evaluation prompts, model scorecards, and evidence endpoints. It is a simple quality plan for customer conversations, not a full benchmark platform or automatic ranking system.

Dimension	Customer question
Task quality	Does the model answer the customer's real use case well?
Reliability	Does the route work repeatedly without confusing errors?
Latency	Is the response fast enough for the customer workflow?
Cost	Is the model affordable under the customer's budget?
Safety	Can the request be sent without exposing sensitive data?
Fallback	What happens if the first provider is unavailable?

Customer Discovery Checklist

/v1/gateway/discovery-checklist helps before promising an OpenRouter-like gateway. It turns a broad customer idea into clear scope by asking about the customer goal, model and provider scope, tenant rules, data governance, commercial reporting, and production expectations. It also lists evidence to collect, red flags, fit assessment, and a recommended first pilot.

Area	What to clarify
Customer goal	What workflow calls the gateway first and who decides pilot success.
Provider scope	Which provider and capabilities must work first.
Tenant rules	Which customers can use which models, budgets, and keys.
Data rules	What can be logged, retained, redacted, or deleted.
Production expectation	Whether this is a demo, pilot, limited production, or full production.

Proposal Summary

/v1/gateway/proposal-summary turns discovery into a simple customer-facing scope note. It explains the customer problem, recommended positioning, phase one scope, what is not in phase one, customer deliverables, decision points, main risks, and recommended next steps. It is not a legal quote or final contract.

Part	Purpose
Phase one scope	Show what the first pilot should include.
Not in phase one	Avoid promising marketplace, SLA, billing, or many providers too early.
Decision points	Help the customer choose custom gateway, first provider, mock/live test, and pilot/production path
Risks	Explain provider differences, data handling, and prototype readiness clearly.
Next steps	Move from discussion to one small technical pilot.

Recommended Roadmap

Phase	Goal	Result
1. Current prototype	Qwen-only gateway with mock dashboard	Customer can understand the concept
2. Live Qwen demo	Use Model Studio API key for real chat	Validate latency and response quality
3. Gateway controls	Persist logs, usage, model access rules, budgets, and BYOK mapping	Better customer management
4. More providers	Enable OpenAI, Claude, Xiaomi adapters	More complete OpenRouter-like direction
5. Production	Database, secrets, streaming, fallback, billing	Enterprise-ready platform path

Reference Documents

OpenRouter API Reference: https://openrouter.ai/docs/api/reference/overview/

OpenRouter Authentication: https://openrouter.ai/docs/api-keys

OpenRouter Limits: https://openrouter.ai/docs/api-reference/limits/

OpenRouter Models API: https://openrouter.ai/docs/api/api-reference/models/get-models

OpenRouter Model Fallbacks: https://openrouter.ai/docs/guides/routing/model-fallbacks

OpenRouter Provider Routing: https://openrouter.ai/docs/guides/routing/provider-selection/

OpenRouter BYOK: https://openrouter.ai/docs/use-cases/byok/

Alibaba Cloud Model Studio DashScope API Reference: https://www.alibabacloud.com/help/doc-detail/3016809.html

Final Recommendation

Start small and explain clearly. Use the visual dashboard to show the customer-facing API, the routing layer, and the provider adapter. Use mock mode for discussion and live Qwen mode only when real model testing is needed. Add more providers only after the customer understands the value of the gateway.